



Universitat d'Alacant
Universidad de Alicante

Práctica 01

Sesión 2 - Implementación del algoritmo backtracking



Nombre: Daironas Sukis

NIE: Y1759578T

Grupo: 01

Asignatura: Sistemas Inteligentes, Ingeniería Informática, Universidad de Alicante

1. Contexto.....	3
2. Modificación del variable.py.....	3
3. Modificación del main.py.....	3
3.1 Función encuentraRestricciones.....	3
3.2 Función isFeasible.....	4
3.3 Función BackTracking.....	5
3.4 pintarSolucion.....	6
3.5 Modificación de la función main.....	6
4. Ejecución del algoritmo BackTracking.....	7

1. Contexto

En esta sesión se pide implementar el algoritmo de *backtracking*, para ello primero necesitamos implementar una función, *encuentraRestricciones* en el que se hallen las restricciones de las variables, es decir, comprobar si dada una variable se cruza con alguna otra, restringiendo el uso de distintas posibles palabras. Posteriormente, se implementará la función *isFeasible*, con el objetivo de comprobar si una palabra es candidata a pertenecer a dicha variable. La función de *BackTracking*, que usará la función *isFeasible*, consistirá en el propio algoritmo para probar todas las combinaciones necesarias con el objetivo de encontrar la solución del crucigrama. Una vez encontrada la solución, se usará la función *pintarSolucion* para rellenar visualmente el crucigrama resuelto.

2. Modificación del variable.py

Para esta sesión vamos a necesitar un nuevo atributo de la clase *Variable* que represente el estado de la variable. Si la variable está libre y no tiene ninguna palabra asignada, entonces el valor de la variable *valor* será **None**. Si la variable está ocupada, contendrá el *String* de la palabra ocupada. Este atributo no era necesario en la *Sesión 01* debido a que no existía aún un algoritmo que pudiese resolver el crucigrama, y tuviese asignar a las variables un valor, sin embargo en esta *Sesión 02* es necesario guardar valores y comprobar si palabra está siendo utilizada por otra variable.

```
class Variable:
    def __init__(self, nombre, tipo, fila, col, longitud, dominio):
        self.nombre = nombre
        self.tipo = tipo          # h/v
        self.fila = fila
        self.col = col
        self.longitud = longitud
        self.dominio = dominio
        self.restricciones = []
        self.valor = None
```

3. Modificación del main.py

3.1 Función encuentraRestricciones

Esta función se encarga de encontrar todas las restricciones de una variable.

```
def encuentraRestricciones(listaVariables):
    variablesHorizontales = []
    variablesVerticales = []

    for v in listaVariables:
        if v.tipo == 'H':
            variablesHorizontales.append(v)
        else:
            variablesVerticales.append(v)
```

```

for var_h in variablesHorizontales:
    for var_v in variablesVerticales:
        # CONDICIÓN 1: La variable vertical está posicionada entre la columna inicial
        y la columna final de la variable horizontal
        # CONDICIÓN 2: La variable horizontal está posicionada entre la fila inicial y
        la fila final
        # Si se cumplen las 2 condiciones, las variables se cruzan
        if var_h.col <= var_v.col <= var_h.col + var_h.longitud - 1 and var_v.fila <=
var_h.fila <= var_v.fila + var_v.longitud - 1:
            # En que posición de la variable se cruzan
            pos_en_h = var_v.col - var_h.col
            pos_en_v = var_h.fila - var_v.fila

            var_h.restricciones.append((var_v, pos_en_h, pos_en_v))
            var_v.restricciones.append((var_h, pos_en_v, pos_en_h))

```

Primero separamos las variables en dos listas, horizontales y verticales, y recorremos el primer bucle for para guardar en cada una de estas listas la variable correspondiente. El siguiente bucle for anidado nos permite recorrer todos los casos para cada una de las variables sin que se repitan las restricciones.

Una vez que tenemos la variable horizontal y la variable vertical, tenemos que comprobar si se cruzan entre ellas, y para eso tenemos que comprobar 2 cosas.

1. Como la variable vertical siempre va a estar posicionada en la misma columna, tenemos que comprobar que la posición inicial de la variable horizontal esté en la misma columna o en una columna de la izquierda, y que la posición final esté en la misma columna o en una de las columnas de la derecha.
2. Como la variable horizontal va a estar en la misma fila siempre, tenemos que comprobar que la posición inicial de la variable vertical esté en la misma fila o en una de las filas de arriba, y que la posición final esté en la misma fila o en una de las filas de abajo.

Con estas dos condiciones cumplidas, podemos asegurar que las variables se cruzan. Al cumplirla, guardamos para la variable horizontal, y para la variable vertical, en que número de casilla choca cada una, y la añadimos a la lista de restricciones como una tupla que guarda con la variable que se choca, el número de celda en que se choca, y el número de celda que se choca el vecino.

3.2 Función isFeasible

Esta función determina si una palabra candidata puede asignarse a una variable concreta sin generar conflictos. Primero comprueba que la palabra no esté siendo utilizada en otra variable distinta del crucigrama. Posteriormente recorre todas las restricciones de la variable, y para cada variable vecina que tenga, comprueba si tiene asignada una palabra, y si es así, comprueba que en la celda que se cruzan coincida justo con el carácter. Si alguna de estas comprobaciones no tienen éxito, simplemente la función devuelve *False*, dado a entender que dicha palabra no se puede meter en dicha variable.

```
def isFeasible(variable, palabra, listaVariables):

    # Si la palabra pasada por parámetro esta siendo utilizada en otra
    variable, FALSE

    for var in listaVariables:
        if var != variable:
            if var.valor == palabra:
                return False

    for restriction in variable.restricciones:
        # El vecino si tiene una palabra puesta
        if restriction[0].valor is not None:
            if palabra[restriction[1]] !=
restriction[0].valor[restriction[2]]:
                return False

    return True
```

3.3 Función BackTracking

Se trata de una función recursiva en el que tiene como parámetro la variable *k*, para contar en qué índice de *listaVariables* se encuentra. El caso base de la función recursiva es que el índice ya ha recorrido toda la *listaVariables*, y se ha encontrado solución con BackTracking.

Para el caso general, comprobamos que para todo dominio, es decir, para todas las posibilidades de palabra que entran en el crucigrama, de la variable que estamos comprobando, si es factible. Si lo es, asignamos a la variable su respectivo *valor*, y realizamos la llamada recursiva con el índice actualizado. Si el backTracking no ha encontrado una solución con esa palabra, esa palabra se descarta y se sigue con la siguiente palabra.

Si la llamada recursiva no encuentra una solución completa a partir de esa asignación, se deshace la designación y se prueba con el siguiente valor del dominio. Si se agotan todos los valores del dominio sin éxito, la función devolverá False, indicando a la variable anterior en la recursión de que pruebe otro valor distinto.

```
def backTracking(k, listaVariables):
    if k == len(listaVariables):
        return True
    else:
        actualVariable = listaVariables[k]
        for dom in actualVariable.dominio:
```

```

        if isFeasible(actualVariable, dom, listaVariables):
            actualVariable.valor = dom
            if backTracking(k+1, listaVariables):
                return True
            actualVariable.valor = None
    return False

```

3.4 pintarSolucion

Esta función recorre toda la lista de variables, imprimiendo en la tabla cada carácter correspondiente al atributo valor, y en su respectiva posición.

```

def pintarSolucion(listaVariables, tablero):
    for var in listaVariables:
        for i in range(var.longitud):
            if var.tipo == 'H':
                tablero.setCelda(var.fila, var.col+i, var.valor[i])
            else:
                tablero.setCelda(var.fila+i, var.col, var.valor[i])

```

3.5 Modificación de la función main

Se ha realizado una modificación en el bloque correspondiente al botón BK para que al pulsarlo, se cree la todas las variables del crucigrama, y se ejecute el algoritmo de backtracking.

```

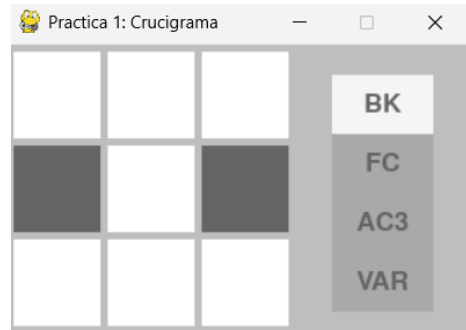
        print('BK')
        variablesHorizontales, contador = construyeVariablesHorizontal(tablero,
almacen)

        variablesVerticales = construyeVariablesVertical(tablero, almacen, contador)
        todasVariables = variablesHorizontales + variablesVerticales
        encuentraRestricciones(todasVariables)
        if backTracking(0, todasVariables):
            pintarSolucion(todasVariables, tablero)
        else:
            MessageBox.showwarning("Alerta", "No hay solución")

```

4. Ejecución del algoritmo BackTracking

Con un tablero 3x3, y el fichero d0.txt que viene con la práctica, si tapamos algunas celdas.



Ejecutando el algoritmo de BackTracking tenemos el siguiente resultado.



Con un tablero de 5x4, y el fichero d0.txt, con el carácter *L* en esa posición, el BackTracking no encuentra ninguna solución y se queda así.

